

From physics to a learned function

A guided notebook companion for Sarel Ye's one-dimensional cuboid experiment.

Prepared 14 September 2026.

Your question: What exactly is the model, which numbers can it learn, and how do we choose the error it should minimize? This guide develops those choices through your translation experiment, with special attention to small neural networks.

The task we are learning

Given the current motion and a force held constant for a fixed interval, predict the cuboid's position and velocity at the end of that interval. Orientation stays fixed. The floor is horizontal and uniform; motion is constrained to one dimension, without tipping or impacts.

current state + applied force + known conditions

→ future position and velocity

Where the repository currently stands

The inspected physics.py computes total push-and-coast displacement for a positive push starting from rest. That is a useful analytical case, but it does not yet provide arbitrary state transitions over a fixed interval. A verified transition reference is needed before generating training labels. Some status text in README.md predates the present code.

How to use these pages

Read one lesson, work its checkpoint on paper, and explain the result in your own words. Later lessons are a map, not a requirement to implement everything now. There is no implementation code here; you remain the author of the simulation and model.

Object	Its job
Physics reference	Calculate future motion from explicit equations; provide checked training labels.
transmodel	Learn a mapping that approximates those future states.
Controller / policy	Choose forces to reach a target. This is a later, separate task.

Simulator labels are reference answers under your chosen equations. Agreement with them does not establish agreement with real hardware.

Define the prediction contract

A model can only learn a well-defined relationship when its inputs contain enough information.

$$\begin{aligned}s_t &= (x_t, v_t) \\ p &= (m, \mu_s, \mu_k, g) \\ s_{t+\Delta t}^{\text{pred}} &= f_{\theta}(s_t, F_t; p)\end{aligned}$$

The state s contains position x in meters and signed velocity v in meters per second. The action F is the signed applied force in newtons. The physical conditions p contain mass, static friction, kinetic friction, and gravity. The superscript pred means a prediction; θ denotes the adjustable model weights. The semicolon separates conditions for readability.

Choose what stays fixed

For your first dataset, keep mass, friction, gravity, and the prediction interval fixed. Then f only needs x , v , and F as inputs. It learns behavior for those particular conditions. Later, if mass or friction varies and is known, include it as an input and vary it in the training experiments. A constant-valued input cannot teach sensitivity to that variable.

Why the action interval matters

Specify that F remains constant throughout Δt . Two experiments can have the same current force but different futures if one releases that force halfway through. If you later allow force changes within an interval, the model must receive enough information about that schedule. A variable prediction interval must also be represented explicitly.

What counts as sufficient state?

For ideal Coulomb friction on a uniform surface, current velocity, force, and the stated conditions determine the next motion under a specified rest rule. If friction varies secretly between experiments, identical visible inputs can produce different answers. More network layers cannot recover information that is absent; you need known parameters, motion history, or an uncertainty model.

Notebook checkpoint

Write the units of every input and output. Choose an illustrative Δt , such as 0.1 s, and write one sentence explaining what the applied force does during that interval. Explain why target position is unnecessary for predicting motion under a supplied force.

Build one trustworthy example

The network needs answers at a fixed future time, not only the final stopping position.

Use $x = 0$ m, $v = 0$ m/s, $m = 2$ kg, $F = 10$ N, $\mu_s = 0.4$, $\mu_k = 0.2$, $g = 9.8$ m/s², and $\Delta t = 0.1$ s. Hold the force throughout the interval.

$$\text{static threshold} = \mu_s mg = 7.84 \text{ N}$$

$$\text{sliding friction magnitude} = \mu_k mg = 3.92 \text{ N}$$

$$a = (10 - 3.92) / 2 = 3.04 \text{ m/s}^2$$

The applied force exceeds the static threshold, so the cuboid begins sliding in the positive direction. Acceleration is constant in this particular interval.

$$v_{\text{next}} = v + a\Delta t = 0.304 \text{ m/s}$$

$$x_{\text{next}} = x + v\Delta t + \frac{1}{2}a(\Delta t)^2 = 0.0152 \text{ m}$$

Why one example is not enough

Training also needs examples at rest, below and near the static threshold, already sliding, coasting, braking, and stopping. Signed velocity matters: kinetic friction opposes motion, not always the positive x direction. At rest, static friction balances the applied force up to its limit.

A critical boundary: stopping inside the interval

A moving cuboid can stop before Δt ends. Continuing the old friction acceleration past that instant can create an unphysical reversal. Resolve the stopping instant and then apply the rest rule; if a sufficiently strong applied force drives a reversal, the later motion must use the new direction. The prediction interval can be larger than the reference simulator's internal numerical step.

Notebook checkpoint

Recalculate the example with $F = 5$ N from rest. Then consider $F = 0$ with positive initial velocity: which direction does acceleration point, and what happens once velocity reaches zero? Check answers in Lesson 10 after your attempt.

Start with an ordinary function

A learned model is a family of possible functions. Training selects one member by adjusting its coefficients.

$$\begin{aligned}x_{\text{next}}^{\text{pred}} &= a_1x + a_2v + a_3F + b_x \\v_{\text{next}}^{\text{pred}} &= c_1x + c_2v + c_3F + b_v\end{aligned}$$

This affine model has eight trainable parameters: six coefficients and two biases (offsets). In compact notation, $y^{\text{pred}} = Wz + b$, where $z = (x, v, F)$, W has two rows and three columns, and b has two entries. Each row computes one output. If using raw physical quantities, coefficients carry the units needed to make each equation consistent.

What training changes

You choose the functional form. Training chooses the numerical values of its coefficients. An initial model may respond too weakly to force; its errors can push the force coefficients upward. The learned coefficients need not correspond one-to-one to named physical quantities.

Why a linear model is useful but limited

In a region where the cuboid keeps sliding in one direction with fixed friction, the constant-acceleration transition is affine in x , v , and F . But one global affine function cannot represent all the changes between sticking, positive sliding, negative sliding, and stopping. That makes it a useful baseline and a reason to investigate nonlinear models.

Choice	Meaning in this experiment
Function family	Affine mapping, neural network, or a correction to known physics.
Trainable parameters	Coefficients, weights, and biases fitted from examples.
Hyperparameters	Choices such as hidden width and learning rate, assessed on validation data.
Physical parameters	Mass and friction; specified conditions, not automatically learned network weights.

Notebook checkpoint

Imagine increasing only a_3 . Which predicted quantity changes when force is positive? Why does this alone not solve the static-friction threshold?

Inside a small neural network

A hidden layer constructs several adjustable responses to the same input, then combines them into predictions.

$$\begin{aligned} q &= W_1 z + b_1 \\ h &= \phi(q) \\ y^{\text{pred}} &= W_2 h + b_2 \end{aligned}$$

Each hidden neuron first forms a weighted sum q_i of the inputs plus its own bias. An activation function ϕ bends that response. The output layer combines the hidden responses. The hidden values are calculated features, not additional measurements, and individual neurons need not have simple physical meanings.

Layer or quantity	Example shape	Adjustable numbers
Input z	3 values: x, v, F	0
W_1 and b_1	4 x 3 weights; 4 biases	16
Hidden h	4 computed values	0 additional
W_2 and b_2	2 x 4 weights; 2 biases	10
Total	3 inputs $\rightarrow$ 4 hidden $\rightarrow$ 2 outputs	26

Follow one neuron numerically

For a toy illustration using dimensionless, scaled inputs $z = (0, 0, 1)$, give one neuron weights (0.2, -0.5, 0.8) and bias -0.1. Its weighted sum is 0.7. With ReLU, its hidden value is 0.7; with tanh, it is about 0.604. The output weights decide how this response contributes to each predicted quantity. These are illustrative weights, not trained results.

Why the activation is essential

Without a nonlinear activation, multiple affine layers collapse into a single affine mapping. Merely adding layers would not overcome the limitation from Lesson 4. Nonlinearity allows different responses in different parts of the input space.

Notebook checkpoint

Using the same toy neuron, change the scaled force input from 1 to 0. Its weighted sum becomes -0.1. What does ReLU output? What does this suggest about a neuron becoming active only in part of the input range?

Choose the shape of transmodel

Use the structure of your physical task to decide what the network should learn.

First exploit the uniform surface

On an infinite, uniform floor, moving the entire experiment from $x = 0$ to $x = 100$ does not change its displacement or velocity change. Absolute position is unnecessary for those changes. This assumption fails near walls, boundaries, or position-dependent friction.

$$(\Delta x^{\text{pred}}, \Delta v^{\text{pred}}) = g_{\theta}(v, F) \text{ [fixed physical conditions]}$$

$$x_{\text{next}}^{\text{pred}} = x + \Delta x^{\text{pred}}$$

$$v_{\text{next}}^{\text{pred}} = v + \Delta v^{\text{pred}}$$

The full world model still receives and returns the state. Only its learned subfunction g omits x . Learning increments builds in translation symmetry and avoids spending capacity on copying absolute position. It does not automatically enforce correct stopping, energy behavior, or consistency between the two outputs.

Function choice	Benefit	Limit / when to use
Direct next-state network	Simple starting definition; flexible	Can learn unwanted dependence on absolute position.
Increment network	Uses the uniform-floor symmetry	Recommended first neural experiment here.
Learn acceleration, then integrate	Connects closely to mechanics	A single constant acceleration can miss a stop within the interval.
Physics + learned correction	Retains known structure	Useful when a measured mismatch justifies the correction.

A concrete starting candidate, not an optimum

Use two scaled inputs (v, F), one hidden layer with 16 ReLU units, and two unrestricted linear outputs for scaled increments. This has 82 parameters: $16 \times 2 + 16 + 2 \times 16 + 2$. Choose it as a small, inspectable starting experiment; compare widths such as 8, 16, and 32 only if validation results justify doing so.

When conditions vary

For known variable conditions, extend g to receive m, μ_s , and μ_k ; add g or Δt only if those also vary. Keep adequate data coverage. Start fixed to isolate learning behavior, then extend the input contract deliberately.

Notebook checkpoint

Explain why adding 100 m to current x should add exactly 100 m to predicted next x while leaving predicted next velocity unchanged.

Activations and scaling

A useful network needs both an appropriate response shape and numerical inputs it can learn from.

Activation	Definition / behavior	Implication here
ReLU	$\max(0, q)$; a straight line with a bend at zero	Simple piecewise-linear approximation; a useful first hidden activation.
tanh	Smooth output between -1 and 1	Can represent signed hidden responses; large inputs saturate and learn slowly.
Linear output	No final nonlinear restriction	Allows signed displacement and velocity changes.

Both ReLU and tanh are reasonable hidden-layer candidates. ReLU does not guarantee an exact friction threshold, and tanh can smooth transitions that are physically sharp. A finite network may struggle near regime boundaries. Judge it on those cases explicitly. Avoid a sigmoid output for unrestricted signed motion because it confines outputs to 0 through 1.

Scale inputs before training

Position, velocity, force, and mass have different numerical ranges. Representing inputs using fixed characteristic scales, or training-set means and standard deviations, usually makes optimization easier. For example, a 10 N force becomes 1 when divided by a 10 N scale. Use the identical transformation during later predictions.

$$\text{scaled force} = F / F_{\text{scale}}$$

$$\text{scaled velocity} = v / v_{\text{scale}}$$

Scale outputs and return to physical units

You can train the network to predict dimensionless increments, then multiply by the selected displacement and velocity scales. The final reported state must remain in meters and meters per second. Fit any data-derived scaling statistics on training data only; fitting them on the test set leaks information.

Initialization and learning rate

Network weights usually start with small random values whose scale depends on the layer and activation; common libraries provide standard initializations. Identical initialization can prevent neurons from learning distinct features. The learning rate controls update size: too large can make training unstable, while too small can make progress slow.

Notebook checkpoint

A velocity scale is 0.5 m/s. What scaled value represents -0.25 m/s? If a predicted scaled displacement is 0.3 with a 0.02 m displacement scale, what displacement is returned?

Design the loss around the task

Supervised learning has no universal loss. A loss turns your priorities into a number the optimizer can reduce.

$$e_{x,i} = (x_i^{\text{pred}} - x_i) / s_x$$

$$e_{v,i} = (v_i^{\text{pred}} - v_i) / s_v$$

$$L = (1/N) \sum_{i=1..N} [e_{x,i}^2 + \lambda e_{v,i}^2]$$

N is the number of examples averaged. The reference outputs x and v are labels. The scales s_x and s_v have matching physical units, making errors dimensionless. The nonnegative weight λ changes velocity's importance relative to position. With the same current state, next-state errors equal increment errors, so this loss works for either output representation.

Make the trade-off concrete

Suppose, illustratively, $s_x = 0.01$ m, $s_v = 0.1$ m/s, and $\lambda = 1$. A position error of 0.005 m contributes 0.25; a velocity error of 0.02 m/s contributes 0.04. Their combined loss is 0.29. These scales express a preference; they are not yet established acceptance tolerances for your project.

Choice	When it makes sense
Squared error (MSE)	A clear first choice for deterministic simulator labels; larger mistakes count more strongly.
Absolute error (MAE)	Less dominated by very large errors, but nonsmooth at zero.
Huber loss	Quadratic for small errors and approximately linear for large ones; useful with noisy outliers.
Multi-step loss	Useful when repeated predictions drift despite good one-step results; adds training complexity.

Choose once, inspect, then revise with evidence

Start with scaled squared errors and equal weighting. Keep the definition fixed for the training run. Inspect position and speed errors separately, especially during stopping. Change scales, sampling, or weights when observed failures and task requirements justify it. A low average can conceal rare but important failures.

Notebook checkpoint

Using the illustrative scales above, calculate the loss for a 0.01 m position error and zero velocity error. Can two models have the same total loss but very different stopping behavior? Explain.

How iteration changes the weights

Prediction produces an error; differentiation tells the optimizer how each adjustable number contributed to it.

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L$$

The gradient is the collection of loss derivatives with respect to all weights and biases. The positive learning rate η sets the step size. Backpropagation computes those derivatives efficiently through the chain rule. An optimizer uses them to update the parameters. PyTorch can calculate the derivatives automatically; understanding what they mean remains useful.

One complete update you can calculate

Use dimensionless toy variables: prediction $y^{\text{pred}} = wz$, input $z = 2$, target $y = 3$, and initial weight $w = 1$. Choose $L = (wz - y)^2$. The prediction is 2 and the loss is 1.

$$dL/dw = 2(wz - y)z = -4$$

$$\text{with } \eta = 0.1: w_{\text{new}} = 1 - 0.1(-4) = 1.4$$

$$\text{new prediction} = 2.8; \text{ new loss} = 0.04$$

The negative derivative says that increasing this weight locally reduces the loss. A large network applies the same principle to many connected parameters. An update can help an average batch loss while worsening some examples; it does not guarantee better test performance.

Vocabulary for actual training

A **batch** is a group of examples used for one update. An **epoch** is one pass through the training set.

Backpropagation computes gradients; it is not itself the optimizer. **Adam** is a common optimizer that adapts updates using gradient history. It still requires settings and validation; it does not choose the correct physical model for you.

Where training lives in your Python project

Conceptually, the reference produces examples, the trainable model maps inputs to outputs, and a training process computes losses and updates weights. After training, prediction loads the learned weights and performs only the forward calculation. No policy or RL reward is required for this predictor.

Notebook checkpoint

Repeat the toy update with $\eta = 0.5$. Does the loss improve? Use the answer to explain why iteration needs a sensible step size, not just repeated attempts.

Choose through evidence

A good first world model is the smallest candidate that meets clearly stated prediction needs on unseen experiments.

A focused first experiment

Use a fixed interval and fixed physical conditions, checked transition labels, an increment network with one small hidden layer, and scaled squared errors. Compare it with unchanged-state and constant-velocity baselines. This is a starting hypothesis to test, not a validated model or a reason to discard your physics reference.

Separate fitting from evaluation

Split complete trajectories or experiment conditions into training, validation, and untouched test sets. Neighboring transitions from one trajectory should not be scattered between them. Train weights on training data; choose settings on validation data; use the test set after choices are settled. Report extrapolation outside the training range separately.

Test the future you actually need

Measure position error in meters and velocity error in m/s at one step and at a specified multi-step horizon. In a rollout, feed predictions back as the next inputs while providing the intended forces. Compare the whole trajectory with the reference. Inspect rest drift, stopping time, and friction-driven reversal rather than relying on one averaged score.

Evidence	Possible response
Training and validation errors both remain high	Check labels, scaling, input sufficiency, and optimization before enlarging the network.
Training error low; validation error high	Investigate overfitting and data coverage; consider a smaller model or more varied examples.
One-step errors low; rollout drifts	Inspect accumulated bias and stopping cases; consider multi-step training later.

Checkpoint answers: Lesson 3: 5 N is below 7.84 N, so the body remains at rest. With no force and positive velocity, acceleration is negative until stopping, then zero. Lesson 5: $\text{ReLU}(-0.1) = 0$. Lesson 7: -0.5 and 0.006 m. Lesson 8: loss = 1; equal averages can hide different errors near stopping. Lesson 9: w becomes 3, prediction 6, loss 9; the update overshoots.

Your next notebook page: define the prediction contract, reproduce the 0.1 s physics example, and draw the two-input increment network. Be able to distinguish its 82 trainable parameters from mass, friction, and its learning rate before implementation.

Math and reading, just in time

Learn enough mathematics to explain the next experiment, then use the experiment to strengthen that understanding.

Learn	Use it here	Evidence of understanding
Functions and algebra	Inputs, outputs, coefficients, and units	Explain the difference between f and its parameters.
Vectors and matrices	Layer dimensions and weighted sums	Count the parameters in a $2 \rightarrow 16 \rightarrow 2$ network.
Derivatives and chain rule	Loss gradients and backpropagation	Reproduce the single-weight update on paper.
Averages and held-out data	Loss aggregation and generalization	Explain why test examples must not choose the settings.
Numerical integration	Reference transitions and stopping events	Explain prediction interval versus internal solver step.

Focused resources

Start with one resource at a time. These are established references for the concepts in this guide, not claims that any network configuration has been benchmarked on your project.

3Blue1Brown - Neural Networks. Watch the first lesson for weights and activations, then the gradient-descent lesson. Pause to connect each picture to your two motion outputs.

www.3blue1brown.com/topics/neural-networks

PyTorch - Learn the Basics. After the paper exercises, read Build the Neural Network, Automatic Differentiation, and Optimizing Model Parameters. Translate the terminology to your experiment before writing code.

pytorch.org/tutorials/beginner/basics/intro.html

Dive into Deep Learning. The linear regression and multilayer perceptron chapters connect equations, losses, and model capacity. Read only the parts needed for your current checkpoint.

d2l.ai

Scope and provenance

Prepared from your questions, README.md, and cuboid_translation_x/physics.py as inspected on 14 September 2026. No network has been trained or validated for this guide. Worked numerical examples are analytical illustrations. Suggested architecture, scales, and interval are starting choices to investigate. This document does not change the repository learning plan or simulation implementation.

Keep in mind: prediction loss teaches a model to predict motion. A control reward teaches a policy which actions are useful. You can understand and train the first before taking on the second.